Faster Sampling: Advanced Solvers

Lecturer: Qiang Sun Lecture 08

Contents

1 Faster Sampling: Advanced Solvers **1**

1.1 Motivation and conventions . . . 1

1.2 Forward process, backward process, and connection to flow models introduced in earlier lectures . . . 2

1.3 Gaussian perturbation schedules and prediction targets . . . 3

1.4 Semilinear ODE and exponential integrators . . . 6

1.5 DDIM as exponential Euler . . . 8

1.6 DEIS: multistep exponential integrators . . . 13

1.7 The DPM-Solver family via log-SNR reparameterization . . . 14

1.7.1 DPM-Solver++: stability via x_0-prediction . . . 16

1.7.2 DPM-Solver-v3 (idea-level) . . . 18

1.8 A numerical-analysis viewpoint: What are these solvers? . . . 19

1.9 Time-parallel sampling via ParaDiGMs . . . 19

1.10 Closing remarks . . . 20

1 Faster Sampling: Advanced Solvers

1.1 Motivation and conventions

Flow and diffusion models generate samples by *simulating a differential equation* that transports a simple distribution (noise) to the data distribution. In continuous time:

- **ODE sampling** follows a deterministic trajectory (often fewer steps);
- **SDE sampling** injects randomness (more diversity, usually more steps).

In this lecture we focus on *training-free acceleration*: the model is fixed after training and we only change the *numerical solver* used at inference. The dominant cost is the *number of function evaluations (*NFE*)*, i.e., the number of neural network calls performed by the solver. The definition of NFE is made precise below.

Definition 1 (Number of function evaluations (NFE))

Suppose a sampler uses N time steps and calls the neural network m times per step. Then

$$\mathrm{NFE} \triangleq mN.$$

Multistep methods reuse past evaluations and often have average $m \approx 1$ after a short warm-up. Classifier-free guidance typically doubles NFE because it evaluates the model twice (conditional/unconditional).

Remark 1 (Time convention)

We follow the diffusion/solver literature: $t = 0$ corresponds to *clean data* and $t = 1$ corresponds to *pure noise*. Hence sampling integrates the reverse-time dynamics from $t = 1$ down to $t = 0$. To reduce to the time convention in our previous lectures (noise $\to$ data with increasing time), we can set $t \to 1 - t$. In this section, we stick to the solver convention for consistency with the literature and to avoid notational clutter.

1.2 Forward process, backward process, and connection to flow models introduced in earlier lectures

Consider a forward (noising) SDE on $[0, 1]$:

$$\mathrm{d}X_t = f_t(X_t)\,\mathrm{d}t + g_t\,\mathrm{d}W_t, \qquad X_0 \sim p_{\mathrm{data}}. \tag{1.1}$$

Let p_t denote the density of X_t. Under standard regularity assumptions, the *reverse-time* SDE (integrated from $t = 1$ to $t = 0$) is

$$\mathrm{d}X_t = \Big(f_t(X_t) - g_t^2\,\nabla_x \log p_t(X_t)\Big)\,\mathrm{d}t + g_t\,\mathrm{d}\bar{W}_t, \qquad X_1 \sim p_{\mathrm{init}}, \tag{1.2}$$

and the associated *probability flow ODE (PF-ODE)* with the *same marginals* $\{p_t\}$ is

$$\frac{\mathrm{d}x_t}{\mathrm{d}t} = f_t(x_t) - \tfrac{1}{2}g_t^2\,\nabla_x \log p_t(x_t), \qquad x_1 \sim p_{\mathrm{init}}. \tag{1.3}$$

In practice the score $\nabla_x \log p_t(x)$ is unknown and replaced by a learned model. Here $\bar{W}_t$ denotes a Brownian motion under the reverse-time filtration; one convenient choice is $\bar{W}_t \triangleq W_1 - W_{1-t}$.

Connection to the course convention (noise $\to$ data with increasing time). In earlier lectures we often parameterize the *generative* dynamics forward in time, starting from noise and moving toward data. To translate the solver convention (integrate from $t = 1 \to 0$) into that convention, set

$$\tau \triangleq 1 - t, \qquad Y_\tau \triangleq X_{1-\tau}, \qquad q_\tau \triangleq p_{1-\tau}. \tag{1.4}$$

Then τ increases from 0 to 1 during generation, with $Y_0 \sim p_{\text{init}}$ and $Y_1 \sim p_{\text{data}}$.

Under this reparameterization, the PF-ODE (1.3) becomes

$$\frac{\mathrm{d}Y_\tau}{\mathrm{d}\tau} = -f_{1-\tau}(Y_\tau) + \tfrac{1}{2} g_{1-\tau}^2 \, \nabla_y \log q_\tau(Y_\tau), \tag{1.5}$$

and the reverse-time SDE (1.2) becomes the forward-in-τ generative SDE

$$\mathrm{d}Y_\tau = \Big(-f_{1-\tau}(Y_\tau) + g_{1-\tau}^2 \, \nabla_y \log q_\tau(Y_\tau)\Big) \, \mathrm{d}\tau + g_{1-\tau} \, \mathrm{d}W_\tau. \tag{1.6}$$

If we define the course-style generative drift

$$u_\tau(y) \triangleq -f_{1-\tau}(y) + \tfrac{1}{2} g_{1-\tau}^2 \, \nabla_y \log q_\tau(y), \tag{1.7}$$

then (1.5) is simply

$$\dot{Y}_\tau = u_\tau(Y_\tau),$$

and (1.6) is its stochastic counterpart

$$\mathrm{d}Y_\tau = \big(u_\tau(Y_\tau) + \tfrac{1}{2} g_{1-\tau}^2 \nabla \log q_\tau(Y_\tau)\big)\mathrm{d}\tau + g_{1-\tau}\mathrm{d}W_\tau,$$

matching the structure used in our earlier notes.

Remark 2 (Flow models as a special case)

If $g_t \equiv 0$, then (1.3) reduces to a deterministic flow ODE. In τ-time, (1.5) becomes $\dot{Y}_\tau = -f_{1-\tau}(Y_\tau)$, matching the pure flow setup from earlier lectures.

1.3 Gaussian perturbation schedules and prediction targets

Many diffusion models admit a Gaussian probability path (Gaussian perturbation kernel)

$$x_t \mid x_0 \sim \mathcal{N}(\alpha_t x_0, \, \beta_t^2 I_d), \qquad \alpha_0 = 1, \; \beta_0 = 0, \; \alpha_1 = 0, \; \beta_1 = 1. \tag{1.8}$$

The schedule (α_t, β_t) determines how signal/noise mix over time. When the forward drift is linear, $f_t(x) = \gamma_t x$, one can convert between the SDE coefficients (γ_t, g_t) and the kernel (α_t, β_t).

Proposition 2 (Relating (γ_t, g_t) and (α_t, β_t))

Assume the linear SDE $\mathrm{d}x_t = \gamma_t x_t \, \mathrm{d}t + g_t \, \mathrm{d}W_t$ has conditional law (1.8). Then, for any $\delta > 0$, we have the following relations for all $t \in [0, 1-\delta]$:

$$\gamma_t = \frac{\dot{\alpha}_t}{\alpha_t}, \qquad g_t^2 = \frac{\mathrm{d}}{\mathrm{d}t} \beta_t^2 - 2 \frac{\dot{\alpha}_t}{\alpha_t} \beta_t^2. \tag{1.9}$$

Proof of Proposition 2. We assume throughout that W_t is a standard d-dimensional Brownian motion, that $\gamma : [0, 1-\delta] \to \mathbb{R}$ and $g : [0, 1-\delta] \to [0, \infty)$ are continuous for any $\delta > 0$, and that $\int_0^{1-\delta} g_s^2 \, \mathrm{d}s < \infty$ so the Itô integral below is well-defined. Write $X_t = x_t$.

Step 1: Explicit solution via an integrating factor (Itô product rule). Define the (deterministic) integrating factor

$$\Phi_t \triangleq \exp\Big(\int_0^t \gamma_u \,\mathrm{d}u\Big), \qquad \text{so that} \qquad \dot{\Phi}_t = \gamma_t \Phi_t, \quad \Phi_0 = 1.$$

We use the Itô product rule for semimartingales: if U_t, V_t are (scalar) semimartingales, then

$$\mathrm{d}(U_t V_t) = U_t \,\mathrm{d}V_t + V_t \,\mathrm{d}U_t + \mathrm{d}\langle U, V\rangle_t,$$

see, e.g., Øksendal (2003) (Ch. 4) or Karatzas and Shreve (1991) (Ch. 3).

Let $U_t = \Phi_t^{-1}$ which has finite variation, hence $\langle U, \cdot\rangle \equiv 0$, aka the quadratic covariation term vanishes because Φ_t^{-1} is a smooth, deterministic function. Apply the Itô product rule with $U_t = \Phi_t^{-1}$ and $V_t = X_t$. Since $\mathrm{d}(\Phi_t^{-1}) = -\gamma_t \Phi_t^{-1} \,\mathrm{d}t$, we obtain

$$\begin{aligned}
\mathrm{d}(\Phi_t^{-1} X_t) &= \Phi_t^{-1} \,\mathrm{d}X_t + X_t \,\mathrm{d}(\Phi_t^{-1}) \\
&= \Phi_t^{-1}\big(\gamma_t X_t \,\mathrm{d}t + g_t \,\mathrm{d}W_t\big) \;+\; X_t\big(-\gamma_t \Phi_t^{-1} \,\mathrm{d}t\big) \\
&= \Phi_t^{-1} g_t \,\mathrm{d}W_t.
\end{aligned}$$

Integrating from 0 to t yields

$$\Phi_t^{-1} X_t = X_0 + \int_0^t \Phi_s^{-1} g_s \,\mathrm{d}W_s,$$

and multiplying by Φ_t gives the explicit solution

$$X_t = \Phi_t X_0 + \int_0^t \frac{\Phi_t}{\Phi_s} g_s \,\mathrm{d}W_s. \tag{1.10}$$

Step 2: Matching the conditional mean. Taking conditional expectation given X_0 in (1.10) and using that an Itô integral has mean zero (again a standard fact; see, e.g., Øksendal, Ch. 3), we get

$$\mathbb{E}[X_t \mid X_0] = \Phi_t X_0.$$

By the assumed conditional law (1.8), $\mathbb{E}[X_t \mid X_0] = \alpha_t X_0$, hence

$$\alpha_t = \Phi_t.$$

Differentiating $\alpha_t = \exp(\int_0^t \gamma_u \,\mathrm{d}u)$ gives

$$\gamma_t = \frac{\dot{\alpha}_t}{\alpha_t}.$$

Step 3: Matching the conditional variance. Conditioned on X_0, the random part of X_t is the Itô integral

$$Z_t \triangleq \int_0^t \frac{\Phi_t}{\Phi_s} g_s \,\mathrm{d}W_s.$$

Since the integrand is deterministic (depends only on time), $Z_t \mid X_0$ is Gaussian with mean 0. Moreover, by Itô isometry (see, e.g., Øksendal (2003), Ch. 3),

$$\mathrm{Cov}(Z_t \mid X_0) = \left(\int_0^t \left(\frac{\Phi_t}{\Phi_s}\right)^2 g_s^2 \,\mathrm{d}s\right) I_d.$$

Therefore,

$$\mathrm{Cov}(X_t \mid X_0) = \mathrm{Cov}(Z_t \mid X_0) = \left(\Phi_t^2 \int_0^t \frac{g_s^2}{\Phi_s^2}\,\mathrm{d}s\right) I_d.$$

Matching with (1.8), i.e. $\mathrm{Cov}(X_t \mid X_0) = \beta_t^2 I_d$, and using $\Phi_t = \alpha_t$, we obtain

$$\beta_t^2 \;=\; \alpha_t^2 \int_0^t \frac{g_s^2}{\alpha_s^2}\,\mathrm{d}s. \tag{1.11}$$

Step 4: Solving for g_t^2. Differentiate (1.11) with respect to t:

$$\frac{\mathrm{d}}{\mathrm{d}t}\beta_t^2 = 2\alpha_t\dot{\alpha}_t \int_0^t \frac{g_s^2}{\alpha_s^2}\,\mathrm{d}s + \alpha_t^2 \cdot \frac{g_t^2}{\alpha_t^2} = 2\frac{\dot{\alpha}_t}{\alpha_t}\,\beta_t^2 + g_t^2.$$

Rearranging yields

$$g_t^2 = \frac{\mathrm{d}}{\mathrm{d}t}\,\beta_t^2 - 2\frac{\dot{\alpha}_t}{\alpha_t}\,\beta_t^2,$$

which is exactly (1.9). □

Score vs. noise prediction. There are generally multiple equivalent ways to parameterize the learned model, e.g., directly regressing the score $\nabla_x \log p_t(x)$ or regressing the noise ε in the Gaussian kernel (1.8). For (1.8), the conditional score is $\nabla_x \log p_t(x \mid x_0) = -(x - \alpha_t x_0)/\beta_t^2$. Writing $x_t = \alpha_t x_0 + \beta_t \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, I_d)$, we get the marginal identity

$$\varepsilon^*(x_t, t) \triangleq \mathbb{E}[\varepsilon \mid x_t], \qquad \nabla_x \log p_t(x_t) = -\frac{1}{\beta_t}\,\varepsilon^*(x_t, t). \tag{1.12}$$

This explains why *ε-prediction* is often numerically more stable than direct score regression: as $t \to 0$, $\beta_t \to 0$ so the score can blow up like β_t^{-1} while ε^* typically remains $\mathcal{O}(1)$.

Other targets: data and velocity prediction. Recall that we assume the Gaussian probability path (1.8), i.e.

$$X_t \;=\; \alpha_t X_0 + \beta_t \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I_d) \text{ independent of } X_0, \tag{1.13}$$

so that $p_t(\cdot \mid X_0 = x_0) = \mathcal{N}(\alpha_t x_0, \beta_t^2 I_d)$.

Besides ε-prediction, common alternatives are to predict the clean data x_0 or the velocity $v_t \triangleq \dot{\alpha}_t x_0 + \dot{\beta}_t \varepsilon$:

$$x_{0,\theta}(x_t, t) \approx \mathbb{E}[x_0 | x_t] = x_0^*(x_t, t), \qquad \blacktriangleright \text{ data / } x_0\text{-prediction} \tag{1.14}$$

$$v_\theta(x_t, t) \approx \mathbb{E}[\dot{\alpha}_t x_0 + \dot{\beta}_t \varepsilon | x_t] =: v^*(x_t, t), \qquad \blacktriangleright \text{ velocity / } v\text{-prediction} \tag{1.15}$$

where the *oracle velocity predictor* can be written in terms of posterior means as

$$v^*(x_t, t) = \dot{\alpha}_t\, x_0^*(x_t, t) + \dot{\beta}_t\, \varepsilon^*(x_t, t). \tag{1.16}$$

Remark 3 (Relation to the common v-prediction target $\alpha_t\varepsilon - \beta_t x_0$)

If the schedule is normalized so that $\alpha_t^2 + \beta_t^2 = 1$, we may write $\alpha_t = \cos\theta_t$ and $\beta_t = \sin\theta_t$ for some angle θ_t. Then

$$\dot{\alpha}_t x_0 + \dot{\beta}_t \varepsilon = \dot{\theta}_t\big(\alpha_t\varepsilon - \beta_t x_0\big). \tag{1.17}$$

Thus predicting $\alpha_t\varepsilon - \beta_t x_0$ differs from predicting $\dot{\alpha}_t x_0 + \dot{\beta}_t\varepsilon$ only by the known scalar factor $\dot{\theta}_t$.

Why $v_t = \dot{\alpha}_t x_0 + \dot{\beta}_t\varepsilon$ is called the velocity. The quantity $\dot{\alpha}_t x_0 + \dot{\beta}_t\varepsilon$ is literally the time-derivative of the noising trajectory (1.13) when (x_0, ε) are held fixed.

Derivation of the velocity target. Fix $x_0 \in \mathbb{R}^d$ and $\varepsilon \in \mathbb{R}^d$ and consider the deterministic curve

$$x_t \;\triangleq\; \phi_t(\varepsilon \mid x_0) \;\triangleq\; \alpha_t x_0 + \beta_t\varepsilon. \tag{1.18}$$

If $\varepsilon \sim \mathcal{N}(0, I_d)$, then x_t has conditional law $x_t \mid x_0 \sim \mathcal{N}(\alpha_t x_0, \beta_t^2 I_d)$, i.e. it realizes the Gaussian kernel.

Differentiating (1.18) in time gives the instantaneous tangent (velocity) along this curve:

$$\frac{\mathrm{d}}{\mathrm{d}t} x_t = \dot{\alpha}_t x_0 + \dot{\beta}_t\varepsilon. \tag{1.19}$$

Equivalently, one may express this velocity as a vector field in the current state x and the conditioning x_0. When $\beta_t \neq 0$, we can solve for the noise as $\varepsilon = (x - \alpha_t x_0)/\beta_t$ and substitute into (1.19) to obtain

$$u_t(x \mid x_0) \triangleq \dot{\alpha}_t x_0 + \dot{\beta}_t \frac{x - \alpha_t x_0}{\beta_t} = \frac{\dot{\beta}_t}{\beta_t}\, x + \Big(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t\Big) x_0. \tag{1.20}$$

Evaluating at $x = x_t = \alpha_t x_0 + \beta_t\varepsilon$ recovers $u_t(x_t \mid x_0) = \dot{\alpha}_t x_0 + \dot{\beta}_t\varepsilon$, so this quantity is exactly the (conditional) velocity of the Gaussian noising trajectory. □

1.4 Semilinear ODE and exponential integrators

Assume the forward drift is linear, $f_t(x) = \gamma_t x$. Substituting (1.12) into the PF-ODE (1.3):

$$\frac{\mathrm{d}x_t}{\mathrm{d}t} = \gamma_t x_t - \tfrac{1}{2} g_t^2\, \nabla_x \log p_t(x_t)$$

yields the *semilinear* ODE

$$\begin{aligned}
\frac{\mathrm{d}x_t}{\mathrm{d}t} &= \gamma_t x_t + \frac{g_t^2}{2\beta_t}\, \varepsilon^*(x_t, t) \\
&= \gamma_t x_t + \frac{\frac{\mathrm{d}}{\mathrm{d}t}\beta_t^2 - 2\frac{\dot{\alpha}_t}{\alpha_t}\beta_t^2}{2\beta_t}\, \varepsilon^*(x_t, t) \\
&= \gamma_t x_t + \left(\dot{\beta}_t - \frac{\dot{\alpha}_t}{\alpha_t}\beta_t\right) \varepsilon^*(x_t, t) \\
&= \gamma_t x_t + b_t\, \varepsilon^*(x_t, t), \qquad \text{where} \;\; b_t \triangleq \alpha_t\, \frac{\mathrm{d}}{\mathrm{d}t}\frac{\beta_t}{\alpha_t}.
\end{aligned} \tag{1.21}$$

The coefficient b_t is known from the schedule; the only learned term is ε_θ. We need the following lemma to derive exponential integrators for (1.21).

Lemma 3 (Variation of constants / Duhamel formula)

Let $\gamma : [0,1] \to \mathbb{R}$ be continuous and define the (scalar) propagator

$$E(s \to t) \;\triangleq\; \exp\Big(\int_s^t \gamma_u \,\mathrm{d}u\Big), \qquad s,t \in [0,1].$$

Consider the possibly nonlinear ODE

$$\dot{x}_t \;=\; \gamma_t x_t + F(x_t, t), \tag{1.22}$$

where $F : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d$ is continuous in t and locally Lipschitz in x so that solutions exist and are unique. Then for any $s,t \in [0,1]$ the solution satisfies

$$x_t = E(s \to t)\, x_s + \int_s^t E(u \to t)\, F(x_u, u)\,\mathrm{d}u. \tag{1.23}$$

In particular, when $s > t$ (integrating backward in time), the same identity holds with an integral over a reversed interval, equivalently

$$x_t = E(s \to t)\, x_s - \int_t^s E(u \to t)\, F(x_u, u)\,\mathrm{d}u.$$

Proof of Lemma 3. We use an integrating factor argument.

Step 1: basic properties of $E(s \to t)$. By definition,

$$E(s \to s) = 1, \qquad \frac{\partial}{\partial t} E(s \to t) = \gamma_t\, E(s \to t), \qquad \frac{\partial}{\partial s} E(s \to t) = -\gamma_s\, E(s \to t).$$

Moreover, E has the semigroup (composition) property: for any r, s, t,

$$E(r \to t) = E(s \to t)\, E(r \to s), \tag{1.24}$$

since both sides equal $\exp(\int_r^t \gamma_u \,\mathrm{d}u)$.

Step 2: integrating factor. Fix $s \in [0,1]$ and define the integrating-factor transform

$$y_t \;\triangleq\; E(t \to s)\, x_t.$$

Differentiate using the product rule (here everything is deterministic, so ordinary calculus applies):

$$\begin{aligned}
\dot{y}_t &= \frac{\mathrm{d}}{\mathrm{d}t} E(t \to s) \cdot x_t + E(t \to s) \cdot \dot{x}_t \\
&= \big(-\gamma_t E(t \to s)\big) x_t + E(t \to s)\big(\gamma_t x_t + F(x_t, t)\big) \\
&= E(t \to s)\, F(x_t, t).
\end{aligned}$$

Integrating from s to t gives

$$y_t - y_s = \int_s^t E(u \to s)\, F(x_u, u)\, \mathrm{d}u.$$

Since $y_s = E(s \to s)x_s = x_s$, we have

$$E(t \to s)\, x_t = x_s + \int_s^t E(u \to s)\, F(x_u, u)\, \mathrm{d}u.$$

Step 3: return to x_t. Multiply both sides by $E(s \to t)$, noting that $E(s \to t)E(t \to s) = 1$, to obtain

$$x_t = E(s \to t)\, x_s + \int_s^t E(s \to t)\, E(u \to s)\, F(x_u, u)\, \mathrm{d}u.$$

Using the semigroup property (1.24) with $(r, s, t) = (u, s, t)$ yields $E(s \to t)\, E(u \to s) = E(u \to t)$, hence

$$x_t = E(s \to t)\, x_s + \int_s^t E(u \to t)\, F(x_u, u)\, \mathrm{d}u,$$

which is (1.23). The case $s > t$ is identical; the integral $\int_s^t$ is simply a negative-oriented integral, equivalently $-\int_t^s$. □

Taking $F(x_u, u) = b_u\, \varepsilon^*(x_u, u)$ in Lemma 3 gives the variation of constants formula for (1.21):

$$x_t = E(s \to t)\, x_s + \int_s^t E(u \to t)\, b_u\, \varepsilon^*(x_u, u)\, \mathrm{d}u. \tag{1.25}$$

Exponential-integrator samplers handle the linear multiplier E exactly and approximate only the integral.

Remark 4 (Why exponential integrators help)

For large steps, a plain Euler update approximates the linear multiplier $E(s \to t)$ by a first-order Taylor expansion, introducing a purely linear distortion. Exponential integrators apply the exact multiplier, so the dominant error comes from approximating the nonlinear integral only.

1.5 DDIM as exponential Euler

We work with a *Gaussian probability path*

$$p_t(\cdot \mid x_0) = \mathcal{N}(\alpha_t x_0, \beta_t^2 I_d),$$

and consider the probability-flow ODE written in the ε-prediction semilinear form

$$\dot{x}_t = \gamma_t x_t + b_t\, \varepsilon_\theta(x_t, t), \qquad t \in [0, 1]. \tag{1.26}$$

For Gaussian schedules, the linear coefficient satisfies $\gamma_t = \dot{\alpha}_t / \alpha_t$, and it is convenient to parameterize the nonlinear coefficient as

$$b_t \;=\; \alpha_t \frac{\mathrm{d}}{\mathrm{d}t}\Big(\frac{\beta_t}{\alpha_t}\Big) \;=\; \dot{\beta}_t - \frac{\dot{\alpha}_t}{\alpha_t}\, \beta_t. \tag{1.27}$$

Propagator. Let $E(s \to t)$ be the propagator of the homogeneous linear ODE $\dot{z}_t = \gamma_t z_t$, i.e.

$$E(s \to t) = \exp\Big(\int_s^t \gamma_u \, \mathrm{d}u\Big).$$

When $\gamma_t = \dot{\alpha}_t/\alpha_t$, we obtain the closed form

$$E(s \to t) = \exp\Big(\int_s^t \frac{\dot{\alpha}_u}{\alpha_u} \, \mathrm{d}u\Big) = \frac{\alpha_t}{\alpha_s}. \tag{1.28}$$

Variation of constants in (α, β)-coordinates. For a denoising step $s > t$ (sampling runs backward from noise to data), the variation-of-constants formula (1.25) applied to (1.26) gives

$$x_t = E(s \to t)\, x_s + \int_s^t E(u \to t)\, b_u\, \varepsilon^*(x_u, u)\, \mathrm{d}u, \tag{1.29}$$

where ε^* denotes the oracle (posterior-mean) noise predictor. Substituting (1.28) and (1.27) into (1.29) yields

$$\begin{aligned} x_t &= \frac{\alpha_t}{\alpha_s} x_s + \int_s^t \frac{\alpha_t}{\alpha_u}\Big(\alpha_u \frac{\mathrm{d}}{\mathrm{d}u}\frac{\beta_u}{\alpha_u}\Big)\varepsilon^*(x_u, u)\, \mathrm{d}u \\ &= \frac{\alpha_t}{\alpha_s} x_s + \alpha_t \int_s^t \frac{\mathrm{d}}{\mathrm{d}u}\Big(\frac{\beta_u}{\alpha_u}\Big)\, \varepsilon^*(x_u, u)\, \mathrm{d}u. \end{aligned} \tag{1.30}$$

Exponential Euler $\Rightarrow$ DDIM. The *exponential Euler* approximation freezes the learned term at the *start of the step* (time s),

$$\varepsilon^*(x_u, u) \;\approx\; \varepsilon^*(x_s, s), \qquad u \in [t, s],$$

and integrates the scalar weight exactly. Applying this to (1.30) gives

$$\begin{aligned} x_t &\approx \frac{\alpha_t}{\alpha_s} x_s + \alpha_t\Big(\int_s^t \frac{\mathrm{d}}{\mathrm{d}u}\Big(\frac{\beta_u}{\alpha_u}\Big)\, \mathrm{d}u\Big)\varepsilon^*(x_s, s) \\ &= \frac{\alpha_t}{\alpha_s} x_s + \alpha_t\Big(\frac{\beta_t}{\alpha_t} - \frac{\beta_s}{\alpha_s}\Big)\varepsilon^*(x_s, s). \end{aligned} \tag{1.31}$$

Replacing the oracle predictor ε^* with a learned network ε_θ yields the deterministic DDIM update (Song et al., 2022):

$$\widetilde{x}_t = \frac{\alpha_t}{\alpha_s}\, \widetilde{x}_s + \alpha_t\Big(\frac{\beta_t}{\alpha_t} - \frac{\beta_s}{\alpha_s}\Big)\varepsilon_\theta(\widetilde{x}_s, s). \tag{1.32}$$

Thus, DDIM is a one-model-call-per-step exponential Euler method applied to the semilinear PF-ODE (1.26).

Corollary 4 (DDIM update in different parameterizations)

Fix a denoising step $s > t$ and let $\widetilde{x}_s$ be the current state. Define either a noise prediction $\widehat{\varepsilon} \approx \varepsilon^*(\widetilde{x}_s, s)$ or a clean prediction $\widehat{x}_0 \approx \mathbb{E}[x_0 \mid x_s = \widetilde{x}_s]$ and let them be related by the consistency relations

$$\widehat{x}_0 = \frac{\widetilde{x}_s - \beta_s \widehat{\varepsilon}}{\alpha_s}, \qquad \widehat{\varepsilon} = \frac{\widetilde{x}_s - \alpha_s \widehat{x}_0}{\beta_s}. \tag{1.33}$$

Algorithm 1: DDIM sampler (PF-ODE; one model call per step)

Require: time grid $1 = t_0 > t_1 > \cdots > t_M = 0$; schedule (α_t, β_t); model $\varepsilon_\theta(x, t)$

Initialize $\widetilde{x}_{t_0} \sim p_{\text{init}}$ `// e.g.` $\mathcal{N}(0, I_d)$

for $i = 0, 1, \ldots, M-1$ **do**

  $s \leftarrow t_i$, $t \leftarrow t_{i+1}$

  $\widehat{\varepsilon} \leftarrow \varepsilon_\theta(\widetilde{x}_s, s)$

  $\widetilde{x}_t \leftarrow \frac{\alpha_t}{\alpha_s}\,\widetilde{x}_s + \alpha_t\Big(\frac{\beta_t}{\alpha_t} - \frac{\beta_s}{\alpha_s}\Big)\widehat{\varepsilon}$

Output: $\widetilde{x}_0$ (approximate sample from p_{data})

Then the deterministic DDIM update $\widetilde{x}_s \mapsto \widetilde{x}_t$ can be written equivalently as

$$\widetilde{x}_t = \frac{\alpha_t}{\alpha_s}\,\widetilde{x}_s + \alpha_t\Big(\frac{\beta_t}{\alpha_t} - \frac{\beta_s}{\alpha_s}\Big)\widehat{\varepsilon} \qquad (\varepsilon\text{-prediction}), \tag{1.34}$$

$$\widetilde{x}_t = \frac{\beta_t}{\beta_s}\,\widetilde{x}_s + \beta_t\Big(\frac{\alpha_t}{\beta_t} - \frac{\alpha_s}{\beta_s}\Big)\widehat{x}_0 \qquad (x_0\text{-prediction}), \tag{1.35}$$

$$\widetilde{x}_t = \alpha_t\,\widehat{x}_0 + \beta_t\,\widehat{\varepsilon} \qquad (\text{reconstruction / } v\text{-view}). \tag{1.36}$$

Proof of Corollary 4. We start from the ε-prediction DDIM update obtained in (1.32) above via exponential Euler:

$$\widetilde{x}_t = \frac{\alpha_t}{\alpha_s}\,\widetilde{x}_s + \alpha_t\Big(\frac{\beta_t}{\alpha_t} - \frac{\beta_s}{\alpha_s}\Big)\widehat{\varepsilon}, \tag{1.37}$$

which is exactly (1.34).

Assume $\alpha_s \neq 0$ and $\beta_s \neq 0$ (true for any interior step under standard noise schedules; not applying to endpoints that creates singularities).

Step 1: consistency relations. We have $x_t = \alpha_t x_0 + \beta_t \varepsilon$. Taking conditional expectation $\mathbb{E}[\cdot|x_t]$ on both sides gives

$$x_t = \alpha_t \mathbb{E}[X_0|x_t] + \beta_t \mathbb{E}[\varepsilon|x_t] = \alpha_t x_0^*(x_t, t) + \beta_t \varepsilon^*(x_t, t),$$

where $x_0^*(x_t, t) = \mathbb{E}[X_0|x_t]$ and $\varepsilon^*(x_t, t) = \mathbb{E}[\varepsilon|x_t]$ are the oracle predictors. Solving this linear relation for either predictor gives the identities

$$x_0^*(x_t, t) = \frac{x_t - \beta_t \varepsilon^*(x_t, t)}{\alpha_t}, \qquad \varepsilon^*(x_t, t) = \frac{x_t - \alpha_t x_0^*(x_t, t)}{\beta_t}. \tag{1.38}$$

In practice, if one has either a noise prediction $\widehat{\varepsilon}$ or a clean prediction $\widehat{x}_0$, one enforces the same consistency relations by defining the other via (1.33), which is exactly (1.38) with $\widehat{x}_0$ and $\widehat{\varepsilon}$ in place of the oracle predictors. Specifically, if a noise prediction $\widehat{\varepsilon}$ is given, then $\widehat{x}_0$ can be computed using:

$$\widetilde{x}_s = \alpha_s \widehat{x}_0 + \beta_s \widehat{\varepsilon},$$

which gives

$$\widehat{x}_0 = \frac{\widetilde{x}_s - \beta_s \widehat{\varepsilon}}{\alpha_s}.$$

Conversely, if $\widehat{x}_0$ is given, solving the same identity for $\widehat{\varepsilon}$ yields the second relation in (1.33):

$$\widehat{\varepsilon} = \frac{\widetilde{x}_s - \alpha_s \widehat{x}_0}{\beta_s}.$$

Step 2: ε-prediction $\Rightarrow$ reconstruction form. Using the definition of $\widehat{x}_0$ above,

$$\begin{aligned}
\alpha_t \widehat{x}_0 + \beta_t \widehat{\varepsilon} &= \alpha_t \Big(\frac{\widetilde{x}_s - \beta_s \widehat{\varepsilon}}{\alpha_s} \Big) + \beta_t \widehat{\varepsilon} \\
&= \frac{\alpha_t}{\alpha_s} \widetilde{x}_s + \Big(\beta_t - \frac{\alpha_t}{\alpha_s} \beta_s \Big) \widehat{\varepsilon} \\
&= \frac{\alpha_t}{\alpha_s} \widetilde{x}_s + \alpha_t \Big(\frac{\beta_t}{\alpha_t} - \frac{\beta_s}{\alpha_s} \Big) \widehat{\varepsilon}.
\end{aligned}$$

The right-hand side matches (1.37), hence $\widetilde{x}_t = \alpha_t \widehat{x}_0 + \beta_t \widehat{\varepsilon}$, which is (1.36).

Step 3: reconstruction $\Rightarrow$ x_0-prediction form. Starting from (1.36) and eliminating $\widehat{\varepsilon}$ using $\widehat{\varepsilon} = (\widetilde{x}_s - \alpha_s \widehat{x}_0)/\beta_s$ (from (1.33)), we obtain

$$\begin{aligned}
\widetilde{x}_t &= \alpha_t \widehat{x}_0 + \beta_t \Big(\frac{\widetilde{x}_s - \alpha_s \widehat{x}_0}{\beta_s} \Big) \\
&= \frac{\beta_t}{\beta_s} \widetilde{x}_s + \Big(\alpha_t - \frac{\beta_t}{\beta_s} \alpha_s \Big) \widehat{x}_0 \\
&= \frac{\beta_t}{\beta_s} \widetilde{x}_s + \beta_t \Big(\frac{\alpha_t}{\beta_t} - \frac{\alpha_s}{\beta_s} \Big) \widehat{x}_0,
\end{aligned}$$

which is (1.35).

Therefore (1.34), (1.35), and (1.36) are equivalent representations of the same deterministic DDIM step, with the predictors related by (1.33). □

Remark 5 (Euler vs. exponential Euler)

DDIM is an *exponential Euler* step for the semilinear PF-ODE in ε-, x_0-, or score-prediction form: it integrates the linear "shrink/scale" factor $E(s \to t) = \alpha_t/\alpha_s$ *exactly* and only approximates the learned nonlinear term. A plain Euler step would also discretize this linear multiplier, which typically degrades stability and accuracy when taking large denoising steps.

In contrast, in a v-prediction parameterization one can often rewrite the PF-ODE so that the drift has no isolated linear part, orequivalently, $E(s \to t) = 1$; see details below. In that case, exponential Euler reduces to ordinary Euler, and DDIM scoincides with a plain Euler update on the corresponding v-ODE.

Why v-prediction makes exponential Euler look like Euler? In ε-prediction, the probability-flow ODE (PF-ODE) is naturally written in a *semilinear* form

$$\dot{x}_t = \underbrace{\frac{\dot{\alpha}_t}{\alpha_t} x_t}_{\text{isolated linear part}} + \underbrace{\left(\dot{\beta}_t - \frac{\dot{\alpha}_t}{\alpha_t}\beta_t\right) \varepsilon^*(x_t, t)}_{\text{learned/nonlinear part}}, \tag{1.39}$$

so the linear propagator is

$$E(s \to t) = \exp\Big(\int_s^t \frac{\dot{\alpha}_u}{\alpha_u}\,\mathrm{d}u\Big) = \frac{\alpha_t}{\alpha_s}.$$

Freezing $\varepsilon^*(x_u, u) \approx \varepsilon^*(x_s, s)$ and integrating the scalar kernel exactly produces the DDIM *exponential-Euler* update, when using $\varepsilon_\theta(x_s, s)$ in place of $\varepsilon^*(x_s, s)$.

By contrast, in the *reconstruction / v-view* one keeps track of the clean/noise decomposition

$$x \approx \alpha_t\, x_0(x, t) + \beta_t\, \varepsilon_\theta(x, t), \qquad \text{where} \qquad x_0(x, t) \triangleq \frac{x - \beta_t \varepsilon_\theta(x, t)}{\alpha_t}.$$

Using the identity $x_t = \alpha_t x_0^*(x_t, t) + \beta_t \varepsilon^*(x_t, t)$, one can rewrite (1.39) as

$$\dot{x}_t = \dot{\alpha}_t\, x_0^*(x_t, t) + \dot{\beta}_t\, \varepsilon^*(x_t, t). \tag{1.40}$$

This representation has *no standalone term proportional to x_t*. Equivalently, if we split the ODE as $\dot{x}_t = 0 \cdot x_t + F(t, x_t)$, then the linear part is zero and therefore the propagator is the identity:

$$E(s \to t) = \exp\Big(\int_s^t 0\,\mathrm{d}u\Big) = 1.$$

Hence, exponential Euler collapses to an Euler-type *freeze the state dependence* rule. Indeed, freezing $x_0^*(x_u, u)$ as $x_0^*(x_s, s) \approx x_0^*(x_u, u)$ and $\varepsilon^*(x_u, u)$ as $\varepsilon^*(x_s, s) \approx \varepsilon^*(x_u, u)$ in (1.40) gives

$$\begin{aligned} x_t &\approx x_s + \int_s^t \dot{\alpha}_u\,\mathrm{d}u\; x_0^*(x_s, s) + \int_s^t \dot{\beta}_u\,\mathrm{d}u\; \varepsilon^*(x_s, s) \\ &= x_s + (\alpha_t - \alpha_s) x_0^*(x_s, s) + (\beta_t - \beta_s)\varepsilon^*(x_s, s) \\ &= \alpha_t\, x_0^*(x_s, s) + \beta_t\, \varepsilon^*(x_s, s) + \big(x_s - \alpha_s x_0^*(x_s, s) - \beta_s \varepsilon^*(x_s, s)\big). \end{aligned}$$

Due to the consistency relation $x_s = \alpha_s x_0^*(x_s, s) + \beta_s \varepsilon^*(x_s, s)$, the residual term vanishes and we obtain

$$x_t = \alpha_t x_0^*(x_s, s) + \beta_t \varepsilon^*(x_s, s),$$

which is exactly the DDIM update in reconstruction form, if we replace the oracle predictors with the learned ones.

A common special case: variance-preserving schedules with $\alpha_t^2 + \beta_t^2 \equiv 1$. If $\alpha_t^2 + \beta_t^2 \equiv 1$, then the PF-ODE in v-prediction form can be rewritten as

$$\begin{aligned} \dot{x}_t &= \gamma_t x_t + \left(\dot{\beta}_t - \frac{\dot{\alpha}_t}{\alpha_t}\beta_t\right) \varepsilon^*(x_t, t) \\ &= \dot{\alpha}_t x_0^*(x_t, t) + \dot{\beta}_t \varepsilon^*(x_t, t) = v^*(x_t, t). \end{aligned}$$

This makes the *no isolated linear part* (and thus $E(s \to t) = 1$) completely explicit: the drift is exactly equal to v^*.

1.6 DEIS: multistep exponential integrators

[check this section with the paper; we may want to add more details on the polynomial fitting and integration steps.]

DDIM is a one-step (first-order) exponential integrator. The Diffusion Exponential Integrator Sampler (DEIS) by Zhang and Chen, 2023 improves accuracy by approximating the integrand in (1.30) with a degree-$(K-1)$ polynomial fitted to the last K model evaluations, then integrating this polynomial *exactly* against the known kernel weight.

Lagrange interpolation. Fix a step index $i \geq K$ and let $\mathcal{T}_i \triangleq \{t_{i-1}, t_{i-2}, \dots, t_{i-K}\}$. Along the numerical trajectory we store $\widehat{\varepsilon}_{t_j} \triangleq \varepsilon_\theta(\widetilde{x}_{t_j}, t_j)$. Define Lagrange basis polynomials on these nodes,

$$\ell_{i,j}(u) \triangleq \prod_{\substack{m=0 \\ m \neq j}}^{K-1} \frac{u - t_{i-1-m}}{t_{i-1-j} - t_{i-1-m}}, \qquad j = 0, 1, \dots, K-1, \tag{1.41}$$

such that $\ell_{i,j}(t_{i-1-k}) = \delta_{j,k}$ for $k = 0, \dots, K-1$, and the interpolant

$$\widehat{\varepsilon}_i(u) \triangleq \sum_{j=0}^{K-1} \ell_{i,j}(u)\, \widehat{\varepsilon}_{t_{i-1-j}}. \tag{1.42}$$

DEIS update. DEIS replaces $\varepsilon_\theta(x_u, u)$ by $\widehat{\varepsilon}_i(u)$ in (1.30):

$$\widetilde{x}_{t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}}\, \widetilde{x}_{t_{i-1}} + \sum_{j=0}^{K-1} C_{i,j}\, \widehat{\varepsilon}_{t_{i-1-j}}, \tag{1.43}$$

with coefficients

$$C_{i,j} \triangleq \alpha_{t_i} \int_{t_{i-1}}^{t_i} \frac{\mathrm{d}}{\mathrm{d}u}\Big(\frac{\beta_u}{\alpha_u}\Big)\, \ell_{i,j}(u)\, \mathrm{d}u. \tag{1.44}$$

The $C_{i,j}$ depend only on the grid and schedule, so they can be precomputed.

DDIM as DEIS with $K = 1$. When $K = 1$, $\ell_{i,0} \equiv 1$ and (1.43) reduces to DDIM.

Algorithm 2: DEIS-K: multistep exponential integrator sampler

Require: grid $1 = t_0 > \cdots > t_M = 0$; history length K; schedule (α_t, β_t); model ε_θ
Initialize $\widetilde{x}_{t_0} \sim p_{\text{init}}$
Warm up for $K-1$ steps using DDIM to obtain $\widetilde{x}_{t_1}, \dots, \widetilde{x}_{t_{K-1}}$
Store $\widehat{\varepsilon}_{t_j} \leftarrow \varepsilon_\theta(\widetilde{x}_{t_j}, t_j)$ for $j = 0, \dots, K-1$
for $i = K, \dots, M$ **do**
  Look up coefficients $\{C_{i,j}\}_{j=0}^{K-1}$ from (1.44)
  $\widetilde{x}_{t_i} \leftarrow \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}}\, \widetilde{x}_{t_{i-1}} + \sum_{j=0}^{K-1} C_{i,j}\, \widehat{\varepsilon}_{t_{i-1-j}}$
  $\widehat{\varepsilon}_{t_i} \leftarrow \varepsilon_\theta(\widetilde{x}_{t_i}, t_i)$

Observation 1 (Why DEIS is efficient)

Single-step k-th order methods typically require multiple model calls per step (e.g., Runge–Kutta stages). DEIS is *multistep*: after a short warm-up, each additional step needs only one new evaluation. Thus the average $m \approx 1$ while achieving higher-order accuracy.

1.7 The DPM-Solver family via log-SNR reparameterization

The DPM-Solver family (Lu et al., 2022, 2023; Zheng et al., 2023) combines two ideas:

1. write the PF-ODE in the semilinear ε-prediction form (1.21);
2. reparameterize time using the *half-log SNR*.

Half-log SNR. For (1.8), define

$$\lambda_t \triangleq \tfrac{1}{2} \log \frac{\alpha_t^2}{\beta_t^2} = \log \frac{\alpha_t}{\beta_t}. \tag{1.45}$$

Then $\mathrm{SNR}(t) \triangleq \alpha_t^2/\beta_t^2 = e^{2\lambda_t}$ and $\beta_t/\alpha_t = e^{-\lambda_t}$. Typical schedules make $t \mapsto \lambda_t$ strictly decreasing.

Observation 2 (Why a uniform grid in λ helps)

A step size $h \triangleq \lambda_t - \lambda_s$ corresponds to a *multiplicative* SNR change: $\mathrm{SNR}(t)/\mathrm{SNR}(s) = e^{2h}$. Thus uniform λ-steps allocate effort according to relative changes in noise level, which often matches denoising difficulty better than a uniform grid in t.

Exact integral equation in λ-time. Let $t_\lambda(\cdot)$ be the inverse map and define $\widehat{x}_\lambda \triangleq x_{t_\lambda(\lambda)}$ and $\widehat{\varepsilon}_\theta(\widehat{x}_\lambda, \lambda) \triangleq \varepsilon_\theta(\widehat{x}_\lambda, t_\lambda(\lambda))$. Then the PF-ODE implies

$$x_t = \frac{\alpha_t}{\alpha_s} x_s - \beta_t \int_{\lambda_s}^{\lambda_t} e^{\lambda_t - \lambda}\, \widehat{\varepsilon}_\theta(\widehat{x}_\lambda, \lambda)\, \mathrm{d}\lambda, \qquad h \triangleq \lambda_t - \lambda_s \geq 0. \tag{1.46}$$

Approximating this exponentially weighted integral with a small number of model evaluations yields fast solvers.

Derivation of (1.46). Starting from the semilinear form (1.21):

$$\frac{\mathrm{d}x_t}{\mathrm{d}t} = \gamma_t x_t + b_t\, \varepsilon_\theta(x_t, t), \qquad b_t = \alpha_t \frac{\mathrm{d}}{\mathrm{d}t}\Big(\frac{\beta_t}{\alpha_t}\Big),$$

define the scaled state $y_t \triangleq x_t/\alpha_t$. Then the linear term cancels and

$$\frac{\mathrm{d}y_t}{\mathrm{d}t} = \frac{\mathrm{d}}{\mathrm{d}t}\Big(\frac{\beta_t}{\alpha_t}\Big)\,\varepsilon_\theta(x_t, t). \tag{1.47}$$

Using $\beta_t/\alpha_t = e^{-\lambda_t}$, we have

$$\frac{\mathrm{d}}{\mathrm{d}t}\Big(\frac{\beta_t}{\alpha_t}\Big)\,\mathrm{d}t = \mathrm{d}\big(e^{-\lambda_t}\big) = -e^{-\lambda}\,\mathrm{d}\lambda,$$

so in λ-time the dynamics become especially simple:

$$\frac{\mathrm{d}}{\mathrm{d}\lambda}\Big(\frac{\widehat{x}_\lambda}{\widehat{\alpha}_\lambda}\Big) = -e^{-\lambda}\,\widehat{\varepsilon}_\theta(\widehat{x}_\lambda, \lambda), \qquad \widehat{\alpha}_\lambda \triangleq \alpha_{t_\lambda(\lambda)}.$$

Integrating from λ_s to λ_t gives the desired integral equation (1.46). □

Equivalently, in λ-time one can write

$$\frac{\mathrm{d}}{\mathrm{d}\lambda}\Big(\frac{\widehat{x}_\lambda}{\widehat{\alpha}_\lambda}\Big) = -e^{-\lambda}\,\widehat{\varepsilon}_\theta(\widehat{x}_\lambda, \lambda), \qquad \widehat{\alpha}_\lambda \triangleq \alpha_{t_\lambda(\lambda)}. \tag{1.48}$$

ϕ-functions (exponential-integrator weights). Writing (1.46) in the shifted variable $r = \lambda - \lambda_s \in [0, h]$ gives

$$x_t = \frac{\alpha_t}{\alpha_s}x_s - \beta_t \int_0^h e^{h-r}\,\widehat{\varepsilon}_\theta(\widehat{x}_{\lambda_s+r}, \lambda_s + r)\,\mathrm{d}r.$$

If one approximates the integrand by a Taylor/polynomial in r, the resulting scalar coefficients are *closed-form* functions of h, known as ϕ-functions:

$$\phi_k(h) \triangleq \int_0^h e^{h-r}\,\frac{r^{k-1}}{(k-1)!}\,\mathrm{d}r, \qquad \phi_1(h) = e^h - 1,\ \phi_2(h) = e^h - 1 - h,\ \phi_3(h) = e^h - 1 - h - \tfrac{1}{2}h^2. \tag{1.49}$$

DPM-Solver methods can be viewed as exponential integrators in λ-time whose quadrature rules are designed around these weights.

DPM-Solver-1 = DDIM. Freezing the integrand in (1.46) at (x_s, λ_s) gives

$$x_t \approx \frac{\alpha_t}{\alpha_s}x_s - \beta_t\big(e^h - 1\big)\,\varepsilon_\theta(x_s, s), \tag{1.50}$$

which is equivalent to (1.32).

DPM-Solver-2 (midpoint). Let $\lambda_{\mathrm{mid}} \triangleq \lambda_s + \frac{1}{2}h$ and $t_{\mathrm{mid}} \triangleq t_\lambda(\lambda_{\mathrm{mid}})$. A practical second-order update uses a midpoint evaluation:

$$x_{\mathrm{mid}} \leftarrow \frac{\alpha_{t_{\mathrm{mid}}}}{\alpha_s}x_s - \beta_{t_{\mathrm{mid}}}\big(e^{h/2} - 1\big)\,\varepsilon_\theta(x_s, s), \tag{1.51}$$

$$x_t \leftarrow \frac{\alpha_t}{\alpha_s}x_s - \beta_t\big(e^h - 1\big)\,\varepsilon_\theta(x_{\mathrm{mid}}, t_{\mathrm{mid}}). \tag{1.52}$$

Algorithm 3: DPM-Solver-2 (midpoint; ε-prediction)

Require: grid $t_0 > t_1 > \cdots > t_M$ (e.g. $t_0 = 1, t_M = 0$); $\lambda_t = \log(\alpha_t/\beta_t)$ and inverse t_λ; model ε_θ

Initialize $\widetilde{x}_{t_0} \sim p_{\text{init}}$

for $i = 1, \ldots, M$ **do**

$s \leftarrow t_{i-1}$, $t \leftarrow t_i$, $h \leftarrow \lambda_t - \lambda_s$

$\lambda_{\text{mid}} \leftarrow \lambda_s + \frac{1}{2}h$, $t_{\text{mid}} \leftarrow t_\lambda(\lambda_{\text{mid}})$

$\widehat{\varepsilon}_s \leftarrow \varepsilon_\theta(\widetilde{x}_s, s)$

$\widetilde{x}_{\text{mid}} \leftarrow \frac{\alpha_{t_{\text{mid}}}}{\alpha_s}\,\widetilde{x}_s - \beta_{t_{\text{mid}}}\big(e^{h/2} - 1\big)\,\widehat{\varepsilon}_s$

$\widehat{\varepsilon}_{\text{mid}} \leftarrow \varepsilon_\theta(\widetilde{x}_{\text{mid}}, t_{\text{mid}})$

$\widetilde{x}_t \leftarrow \frac{\alpha_t}{\alpha_s}\,\widetilde{x}_s - \beta_t\big(e^{h} - 1\big)\,\widehat{\varepsilon}_{\text{mid}}$

Remark 6 (Classical ODE methods hiding underneath)

The representation (1.46) is an exponentially weighted integral equation in λ. DPM-Solver-1 corresponds to exponential Euler; DPM-Solver-2 is an exponential midpoint / Heun-type correction. Higher-order variants use additional intermediate evaluations or multistep differences to approximate the λ-integral more accurately.

1.7.1 DPM-Solver++: stability via x_0-prediction

Classifier-free guidance can greatly increase the magnitude of the model output. In ε-prediction this may lead to unstable updates when using large steps. DPM-Solver++ mitigates this by switching to *x_0-prediction* and (optionally) clipping/thresholding the predicted $\widehat{x}_0$.

Relating x_0- and ε-prediction. From (1.8),

$$\varepsilon_\theta(x_t, t) = \frac{x_t - \alpha_t\, x_{0,\theta}(x_t, t)}{\beta_t}, \qquad x_{0,\theta}(x_t, t) = \frac{x_t - \beta_t\, \varepsilon_\theta(x_t, t)}{\alpha_t}. \tag{1.53}$$

A key simplification in λ-time. As above, view the trajectory in λ-time via $\widehat{x}_\lambda \triangleq x_{t_\lambda(\lambda)}$. Define the corresponding clean predictor $\widehat{x}_{0,\theta}(\widehat{x}_\lambda, \lambda) \triangleq x_{0,\theta}(\widehat{x}_\lambda, t_\lambda(\lambda))$. Starting from (1.48) and using $e^{-\lambda} = \beta/\alpha$ together with (1.53) gives

$$\frac{\mathrm{d}}{\mathrm{d}\lambda}\Big(\frac{\widehat{x}_\lambda}{\widehat{\alpha}_\lambda}\Big) = -\Big(\frac{\widehat{x}_\lambda}{\widehat{\alpha}_\lambda} - \widehat{x}_{0,\theta}(\widehat{x}_\lambda, \lambda)\Big). \tag{1.54}$$

Let $\widehat{y}_\lambda \triangleq \widehat{x}_\lambda/\widehat{\alpha}_\lambda$. Then $\frac{\mathrm{d}\widehat{y}_\lambda}{\mathrm{d}\lambda} = -\widehat{y}_\lambda + \widehat{x}_{0,\theta}(\widehat{x}_\lambda, \lambda)$: an exponential relaxation toward the clean prediction on a unit λ-timescale.

Proposition 5 (Exact integral form in x_0-prediction)

Let $s > t$ and $h \triangleq \lambda_t - \lambda_s \geq 0$. The PF-ODE written as (1.54) implies

$$x_t = \frac{\beta_t}{\beta_s} x_s + \alpha_t \int_{\lambda_s}^{\lambda_t} e^{-(\lambda_t - \lambda)}\, \widehat{x}_{0,\theta}(\widehat{x}_\lambda, \lambda)\, \mathrm{d}\lambda. \tag{1.55}$$

Proof. Equation (1.54) is linear in $\widehat{y}_\lambda$. Solving with an integrating factor gives $\widehat{y}_{\lambda_t} = e^{-h}\widehat{y}_{\lambda_s} + \int_{\lambda_s}^{\lambda_t} e^{-(\lambda_t-\lambda)}\widehat{x}_{0,\theta}(\widehat{x}_\lambda, \lambda)\, \mathrm{d}\lambda$. Multiplying by α_t and using $\alpha_t e^{-h}/\alpha_s = \beta_t/\beta_s$ yields (1.55). □

First-order DPM-Solver++ update. In x_0-prediction, a one-step (first-order) update has the stable form

$$x_t \approx \frac{\beta_t}{\beta_s} x_s + \alpha_t\big(1 - e^{-h}\big)\, x_{0,\theta}(x_s, s), \qquad h = \lambda_t - \lambda_s \geq 0. \tag{1.56}$$

Note that $\beta_t/\beta_s \leq 1$ and $1 - e^{-h} \in (0, 1)$, so coefficients stay bounded even for coarse steps.

Why this helps under strong guidance. Compare the first-order updates in ε-prediction versus x_0-prediction:

$$\text{DDIM / DPM-Solver-1 } (\varepsilon\text{-pred}): \qquad x_t \approx \frac{\alpha_t}{\alpha_s} x_s - \beta_t\big(e^h - 1\big)\, \varepsilon_\theta(x_s, s), \tag{1.57}$$

$$\text{DPM-Solver++-1 } (x_0\text{-pred}): \qquad x_t \approx \frac{\beta_t}{\beta_s} x_s + \alpha_t\big(1 - e^{-h}\big)\, x_{0,\theta}(x_s, s). \tag{1.58}$$

When CFG amplifies the model output, the ε-update can be destabilized by the unbounded factor $e^h - 1$ for coarse steps, while x_0-prediction keeps coefficients bounded and allows additional stabilization by clipping/thresholding $x_{0,\theta}$ directly in data space.

DPM-Solver++-2 (midpoint; 2nd order). Let $\lambda_{\text{mid}} = \lambda_s + \frac{1}{2}h$ and $t_{\text{mid}} = t_\lambda(\lambda_{\text{mid}})$. A midpoint variant mirrors DPM-Solver-2:

$$x_{\text{mid}} \leftarrow \frac{\beta_{t_{\text{mid}}}}{\beta_s} x_s + \alpha_{t_{\text{mid}}}\big(1 - e^{-h/2}\big)\, x_{0,\theta}(x_s, s), \tag{1.59}$$

$$x_t \leftarrow \frac{\beta_t}{\beta_s} x_s + \alpha_t\big(1 - e^{-h}\big)\, x_{0,\theta}(x_{\text{mid}}, t_{\text{mid}}). \tag{1.60}$$

In practice one often applies dynamic thresholding or simple clipping to the midpoint prediction $x_{0,\theta}(x_{\text{mid}}, t_{\text{mid}})$ before using it.

Algorithm 4: DPM-Solver++-2 (midpoint; x_0-prediction)

Require: grid $t_0 > \cdots > t_M$; $\lambda_t = \log(\alpha_t/\beta_t)$ and inverse t_λ; schedule (α_t, β_t); model $x_{0,\theta}(x, t)$

Initialize $\widetilde{x}_{t_0} \sim p_{\text{init}}$

for $i = 1, \ldots, M$ **do**

- $s \leftarrow t_{i-1}$, $t \leftarrow t_i$, $h \leftarrow \lambda_t - \lambda_s$
- $\lambda_{\text{mid}} \leftarrow \lambda_s + \frac{1}{2}h$, $t_{\text{mid}} \leftarrow t_\lambda(\lambda_{\text{mid}})$
- $\widehat{x}_{0,s} \leftarrow x_{0,\theta}(\widetilde{x}_s, s)$
- $\widetilde{x}_{\text{mid}} \leftarrow \frac{\beta_{t_{\text{mid}}}}{\beta_s}\,\widetilde{x}_s + \alpha_{t_{\text{mid}}}\big(1 - e^{-h/2}\big)\,\widehat{x}_{0,s}$
- $\widehat{x}_{0,\text{mid}} \leftarrow x_{0,\theta}(\widetilde{x}_{\text{mid}}, t_{\text{mid}})$
- $\widetilde{x}_t \leftarrow \frac{\beta_t}{\beta_s}\,\widetilde{x}_s + \alpha_t\big(1 - e^{-h}\big)\,\widehat{x}_{0,\text{mid}}$

Multistep variants. DPM-Solver++ also admits multistep versions (2M, 3M, ...) that reuse a short history so that each step needs only one new model evaluation after warm-up, similar in spirit to DEIS.

1.7.2 DPM-Solver-v3 (idea-level)

DPM-Solver-v3 (Zheng et al., 2023) automates a design choice: which prediction target (ε, x_0, v, score, or hybrids) to use when writing the PF-ODE as $\dot{x}_t = \gamma_t x_t + N_\theta(x_t, t)$. It selects a parameterization and one-step coefficients by minimizing a proxy for the local truncation error, evaluated along a cheap surrogate trajectory (e.g., a first-order forecast). The takeaway is that sampler design can be posed as a small regression problem instead of ad hoc tuning.

A simple residual-minimization view. Write the PF-ODE in semilinear form

$$\dot{x}_t = \gamma_t x_t + N_\theta(x_t, t), \tag{1.61}$$

where γ_t is schedule-dependent and N_θ depends on the chosen parameterization. For a step $s \to t$, the exact variation-of-constants identity is

$$x_t = E(s \to t)\, x_s + \int_s^t E(u \to t)\, N_\theta(x_u, u)\, \mathrm{d}u. \tag{1.62}$$

A broad class of one-step updates uses the ansatz

$$\widetilde{x}_t(\beta) \triangleq E(s \to t)\, x_s + \sum_{m=1}^{q} \beta_m\, \Phi_m(s, t)\, N_\theta(x_s, s_m), \tag{1.63}$$

where $\{s_m\} \subset [t, s]$ are evaluation times and $\Phi_m(s, t)$ are known scalar kernel factors (ϕ-functions, integrals of E, etc.). DPM-Solver-v3 chooses β (and, implicitly, the parameterization in N_θ) by minimizing a proxy for the local discretization error, evaluated along a cheap surrogate trajectory.

Concretely, given a predictor trajectory $\bar{x}_u$ (e.g., a first-order forecast), define the residual

$$\mathcal{R}(\beta) \triangleq \int_s^t E(u \to t)\, N_\theta(\bar{x}_u, u)\, \mathrm{d}u \;\; - \sum_{m=1}^{q} \beta_m\, \Phi_m(s, t)\, N_\theta(x_s, s_m). \tag{1.64}$$

Then the coefficients are chosen by a least-squares criterion

$$\beta^* \in \arg\min_{\beta} \mathbb{E}\big[\|\mathcal{R}(\beta)\|_2^2\big], \tag{1.65}$$

and the parameterization (choice of N_θ) is selected by the smallest achievable proxy error.

1.8 A numerical-analysis viewpoint: What are these solvers?

Many diffusion samplers are classical ODE methods applied to either the PF-ODE itself or an exponential-integrator reformulation (often after a time reparameterization). The table below lists common correspondences.

View / parameterization	**Classical method**	**Sampler name (typical)**
v-prediction	Euler	DDIM (often coincides)
ε/ x_0/ score prediction	exponential Euler	DDIM
log-SNR (λ) + ε-pred	exponential midpoint / Heun	DPM-Solver-2
semilinear, multistep	Adams–Bashforth-type EI	DEIS-K
log-SNR + tailored Taylor / ϕ-weights	specialized EI / RK	DPM-Solver family

1.9 Time-parallel sampling via ParaDiGMs

Standard solvers are sequential: computing x_{t_i} requires $x_{t_{i-1}}$. ParaDiGMs accelerates *wall-clock* sampling by trading parallel compute for reduced latency. It performs Picard iterations: evaluate model-dependent increments on a *guessed* trajectory in parallel, then update the whole trajectory via a parallel prefix-scan.

Affine scan formulation. Many solvers on a grid $t_0 > \cdots > t_M$ can be written abstractly as

$$x_{t_i} = A_i\, x_{t_{i-1}} + b_i(x_{t_{i-1}}), \qquad i = 1, \ldots, M, \tag{1.66}$$

where A_i is known from the schedule and $b_i(\cdot)$ contains model evaluations. Given a guess trajectory $\{x_{t_i}^{(k)}\}$, freeze the nonlinear terms $b_i^{(k)} \triangleq b_i(x_{t_{i-1}}^{(k)})$. Then the next iterate solves the linear recurrence

$$x_{t_i}^{(k+1)} = A_i\, x_{t_{i-1}}^{(k+1)} + b_i^{(k)}, \qquad x_{t_0}^{(k+1)} = x_{t_0}. \tag{1.67}$$

Represent each step as an affine map $T_i^{(k)}(x) = A_i x + b_i^{(k)}$. Affine composition is associative; representing $T(x) = Ax + b$ by (A, b), define

$$(A_2, b_2) \otimes (A_1, b_1) \triangleq (A_2 A_1,\ A_2 b_1 + b_2). \tag{1.68}$$

A parallel prefix-scan over $\{(A_i, b_i^{(k)})\}$ computes all compositions in $O(\log M)$ depth, yielding the updated trajectory.

Windowed iterations (practical detail). In practice one often splits the time grid into windows and runs Picard updates window-by-window, keeping window boundaries fixed from the previous iterate. This improves stability and reduces memory while keeping each window scan-parallel.

Algorithm 5: ParaDiGMs (time-parallel Picard iterations; high-level sketch)

Require: grid $t_0 > \cdots > t_M$; base solver in the form (1.66); iterations K
Initialize a rough trajectory $\{x_{t_i}^{(0)}\}_{i=0}^{M}$ (e.g., one coarse DDIM/DPM pass)
Precompute $\{A_i\}_{i=1}^{M}$ from the schedule
for $k = 0, 1, \ldots, K-1$ **do**
  `// (1) evaluate frozen increments in parallel`
  Compute $\{b_i^{(k)}\}_{i=1}^{M}$ in parallel using model calls on $\{x_{t_i}^{(k)}\}$
  `// (2) integrate the affine recurrence via a parallel scan`
  Compute $\{x_{t_i}^{(k+1)}\}_{i=0}^{M}$ from (1.67) using the scan operator (1.68)
Output: $x_{t_M}^{(K)}$

1.10 Closing remarks

Training-free solvers can drastically reduce sampling cost:

- **DDIM**: first-order exponential Euler for the PF-ODE;
- **DEIS**: multistep exponential integrators (higher order with NFE near 1 per step);
- **DPM-Solver / DPM-Solver++**: log-SNR reparameterization and tailored exponential-integrator updates;
- **ParaDiGMs**: complementary *time parallelism* to reduce latency.

These methods make diffusion models far more practical, but they remain iterative. An important next direction is to *learn* a fast solver (distillation / one-step generative models).

References

Karatzas, I. and Shreve, S. E. (1991). *Brownian Motion and Stochastic Calculus*, volume 113 of *Graduate Texts in Mathematics*. Springer, New York, 2 edition.

Lu, C., Zhou, Y., Bao, F., Chen, J., Li, C., and Zhu, J. (2022). DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps.

Lu, C., Zhou, Y., Bao, F., Chen, J., Li, C., and Zhu, J. (2023). DPM-Solver++: Fast Solver for Guided Sampling of Diffusion Probabilistic Models.

Øksendal, B. (2003). *Stochastic Differential Equations: An Introduction with Applications*. Universitext. Springer, Berlin, Heidelberg, 6 edition.

Song, J., Meng, C., and Ermon, S. (2022). Denoising Diffusion Implicit Models.

Zhang, Q. and Chen, Y. (2023). Fast Sampling of Diffusion Models with Exponential Integrator.

Zheng, K., Lu, C., Chen, J., and Zhu, J. (2023). DPM-Solver-v3: Improved Diffusion ODE Solver with Empirical Model Statistics.